DBSHIELD: DataBase Systems Handling for Injection, Exploitation, Leakage, and DoS

Arnav Priyadarshi (23CS30008) Ketan Suman (23CS30027) Krishna Prabu (23CS30028)
Rahul Chandrakant Patne (23CS10084) Shreeraj Kalbande (23CS30025)

1 Problem Statement

Modern web applications depend heavily on databases to store and process sensitive information such as credentials, personal records, academic data, and administrative entries. Thus, it is important to study and understand how to construct secure databases. Ignoring this often leads to insecure design decisions such as unsafe query construction, weak validation of user-controlled input, insufficient server-side access checks, and inadequate controls against computational abuse. When such weaknesses exist in a database-backed system, the consequences can include unauthorized access, leakage of sensitive information, loss of service availability and more.

We present the **DBSHIELD** project, motivated by the need to demonstrate that these issues are not merely theoretical, but can arise in many places during development. Many users and developers underestimate how severely a database system can be affected when core security mechanisms are absent or improperly implemented. Instead of discussing these risks only in abstract terms, we aim to present through this project a direct comparison between two versions of the same application: an *unshielded* system that is intentionally vulnerable for demonstration, and a *shielded* system that includes appropriate safeguards. This side-by-side design makes the role of security measures obvious, and shows how without them the system can be significantly harmed.

The application selected for this project is a mock student ERP system with three roles: *student*, *teacher*, and *admin*. We selected this setting as it is something the intended audience can relate to easily, and reflects a realistic role-based application in which users authenticate, access restricted information, perform role-specific actions, and interact with shared database records. Such a system will naturally expose the central security requirements of authentication, data isolation and resilience. It therefore provides a practical environment in which database related vulnerabilities and their mitigations can be examined in a meaningful way.

We focus on three representative classes of attack. The first is *SQL injection*, which targets the interaction between user input and backend query execution. When queries are not handled safely, SQL injection may allow login bypass, unauthorized reads, and unintended modifications to database contents. The second is *distributed denial of service*, where attackers repeatedly send requests to the app, causing genuine users to be stuck in the queue. The third is *authorization bypass*, where a user attempts to access resources or operations outside the limits of their intended role, particularly in systems that trust client-side verification such as cookies over server-side verification.

We also present our github repository for reproduction - DBSHIELD

2 Methodology

2.1 Overall Study Design

Two functionally similar versions of the same student ERP application are designed and studied:

- **Unshielded system:** A deliberately vulnerable implementation in which protections are absent, to demonstrate the importance of defending against attacks.
- **Shielded system:** A hardened implementation that preserves the same functionality while incorporating defensive mechanisms to prevent attacks.

This paired design helps isolate the effect of security controls. Since both systems serve the same use case and operate on the same general model of data and user roles, differences in observed behavior can

be attributed more clearly to the presence or absence of safeguards rather than to unrelated architectural changes.

2.2 System Under Study

The application modeled in this project is a student ERP system with three user roles:

- **Student:** Accesses personal details, academic information, and self-related records. Can enroll in courses, deregister from courses, but not view or interact with other students.
- **Teacher:** Accesses and updates academic information related to students and courses. Can create courses, modify contents, but cannot modify other teacher's courses.
- **Admin:** manages users, records, and privileged system-level operations.

This role structure is intentionally chosen because it captures a realistic multi-user scenario in which different permissions must be enforced over the same database. It provides a suitable basis for studying authentication flows, role-based restrictions, protected data access, and misuse of sensitive operations.

2.3 Technology Stack

The system uses a web-application stack composed of the following components:

- **Frontend:** React is used to build the user-facing interface, including login pages, dashboards, navigation, and role-specific views.
- **Backend:** FastAPI is used to implement API endpoints, application logic, authentication handling, request processing, and access control logic.
- **Database:** PostgreSQL and SQLite is used as the relational data store for user accounts, role information, academic records, and related entities.

2.4 Unshielded and Shielded Implementations

In both the cases, the FastAPI backend is responsible for retrieving from the database, perform logical operations, checking and communicating the result to the frontend.

In the *Unshielded Implementation*, the raw SQL is directly fed, without any additional checking, to the database to fetch data. This can cause issues such as SQL Injection, DoS and Authorization Bypass, which are discussed later. The result similarly is directly fed to the user, without verification.

In the *Shielded Implementation*, the input from the frontend is processed, checked for potential SQL Injection attacks, checked for DoS attacks, and is only then processed. This avoids issues that used to arise in the unshielded implementation.

2.5 Study Method

For each of the 3 attack types, we evaluate the unshielded model and the shielded model.

- **SQL Injection:** We test on various SQL injections, and obtain the results for both unshielded and shielded implementations. We verify that shielding did produce significant benefits in mitigating SQL Injection Attacks
- **Distributed Denial of Service:** We test both unshielded and shielded against an attacker spamming the login endpoint to deny service to clients. We verify that in the case of shielding the client did not suffer interruptions to service.
- **Authorization Bypass:** We tested on various attacks to the endpoint for accessing unauthorized information. We verified in the case of shielding such attacks were prevented, but in the case of unshielded the attacker got access to sensitive information.

3 System Architecture

The frontend is built using React.js, a JavaScript library. It manages state-management, routing, ensuring user data persists across sessions.

The backend was implemented using FastAPI. It handles communication with frontend and interfacing with database. It contains the parameters for the study - the `ENABLE_SQLI_PROTECTION` toggle, `ENABLE_DDOS_PROTECTION` toggle and `ENABLE_AUTH_BYPASS_PROTECTION` toggle. Each of the attacks are covered in detail below, discussing how they are implemented and the results obtained.

The database layer uses SQLite and PostgreSQL. The core schema consists of a Users table, Roles table, Courses table, Enrollments table, Grades table. This completely simulates the Student ERP system. A detailed ER diagram is given below.

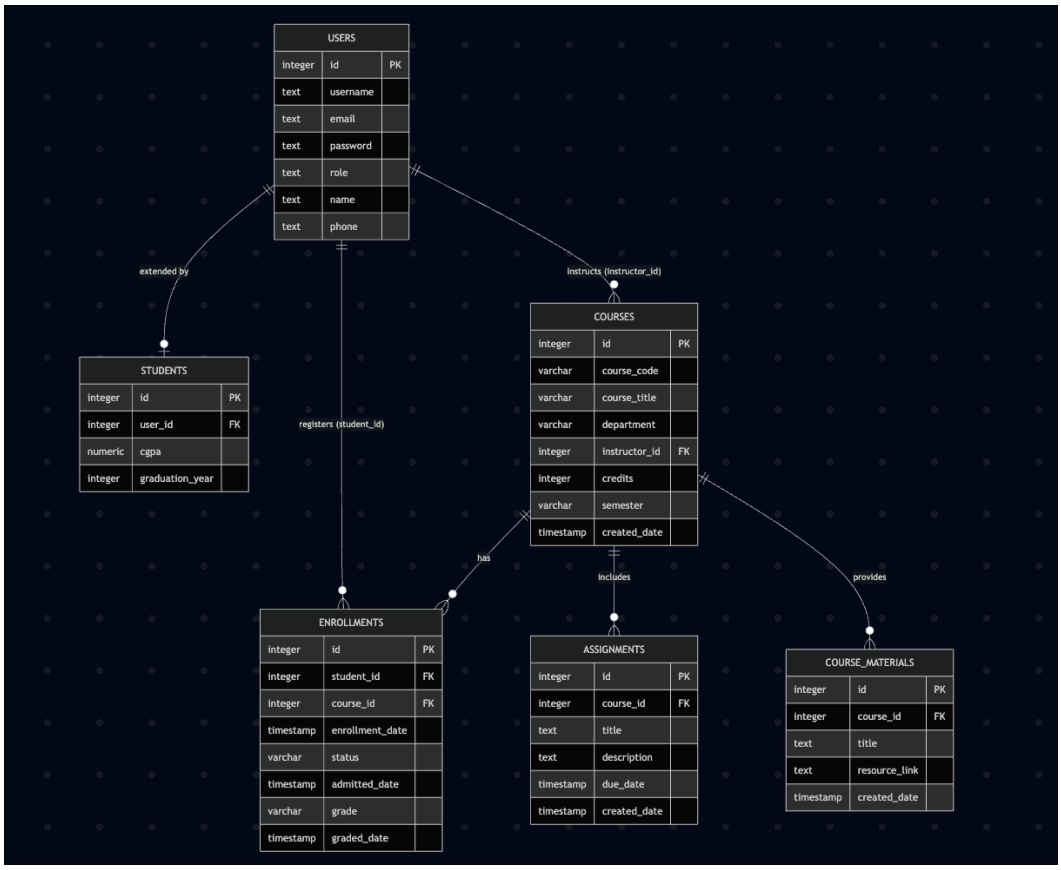

Figure 1: Entity-Relationship diagram mapping users, roles, and academic records.

4 SQL Injection

4.1 Study

The SQL injection study covers both authentication and several ERP data-handling endpoints. The backend preserves an intentionally vulnerable implementation in its original service layer, while the hardened behavior is implemented in a separate module under `sql_injection_prevention`. A single runtime flag, `ENABLE_SQLI_PROTECTION`, determines which handler is used for each request. This paired design is useful for controlled evaluation because the application routes, payload formats, and user-visible functionality remain unchanged, while only the query-construction strategy differs between the two modes.

In the unshielded mode, multiple operations continue to build SQL directly from user-controlled values. These include login, course search, grade and course lookups based on usernames, enrollment and grading workflows, administrative creation and deletion actions, and an especially sensitive admin query endpoint. In the shielded mode, the same operations are routed to secure counterparts that use parameterized queries for user-supplied values. For the admin query interface, which accepts free-form SQL text by design, the secure path does not attempt to parameterize arbitrary statements; instead, it

restricts execution to a single read-only `SELECT` or `WITH` query and blocks mutating or stacked statements.

To evaluate this design, we use a controlled set of payloads against both the login path and representative ERP endpoints. The test corpus includes authentication-bypass inputs based on comments and tautologies, `UNION`-style payloads intended to forge result rows or enumerate schema information, and username or course-code payloads intended to alter selection or deletion behavior in routine ERP operations. Additional tests also check whether values that look like SQL fragments, such as a malicious-looking grade string or username, are stored and handled as literal data in the secure mode rather than being executed as part of the query.

The experiments are executed through the project test harness and through the application toggle that switches between vulnerable and secure handlers. For each case, the main observation is whether the supplied payload changes query semantics, broadens the result set, causes unintended data modification, or bypasses the intended restriction on the admin action endpoint. The comparison therefore measures not only whether login injection is prevented, but also whether the same defensive strategy generalizes across the broader ERP workflow.

Testing procedure. For controlled testing, the backend variable `ENABLE_SQLI_PROTECTION` is switched between `False` and `True` while keeping the same API routes active. A representative login-bypass attempt can be issued against `/api/login` using a SQLite-compatible tautology payload such as `' OR '1'='1`. The vulnerable mode is expected to authenticate the attacker or otherwise alter the intended query logic, whereas the shielded mode is expected to reject the request or treat the supplied string as ordinary input. The same comparison is then repeated across course-search, grade-view, enrollment, and admin-query endpoints using the documented payload set from the project tests.

```
curl -s -X POST http://localhost:8000/api/login \
  -H "Content-Type: application/json" \
    -d "{\"username\":\"' OR '1'='1\",\"password\":\"' OR '1'='1\"}" | jq .
```

When `ENABLE_SQLI_PROTECTION = False`, the expected outcome is a successful login or a query-manipulated response showing that user input has changed SQL semantics. When `ENABLE_SQLI_PROTECTION = True`, the same request is expected to fail safely, either because the payload is detected as malicious or because parameterized handling preserves the intended query structure.

4.2 Results

The updated SQL injection experiments show a consistent separation between the vulnerable ERP service layer and the shielded handlers selected through `ENABLE_SQLI_PROTECTION`. On the login path, the previously documented SQLite-compatible payloads continued to succeed against the unshielded implementation: comment-based and tautology payloads enabled authentication bypass, `UNION`-based payloads could fabricate or expose rows, and blind inference payloads could still alter query logic. This confirms that the vulnerable path remains suitable for controlled demonstration.

The broader ERP tests show the same pattern. In the unshielded mode, username-based injection payloads could broaden student-grade and student-course lookups, and course-search input could alter selection behavior. Administrative actions that interpolate course codes or usernames are likewise exposed in the vulnerable path because the query text is still assembled directly from request data. In contrast, the shielded handlers treat these same payloads as literal values. Search and lookup functions return no unauthorized data, delete operations do not affect unintended rows, enrollment requests using injected usernames fail normally, and malicious-looking values can be stored as plain data without changing query structure.

The hardened admin action endpoint is a particularly important result. Since this interface is meant to accept SQL text, parameterization alone is not sufficient. The secure implementation therefore adds

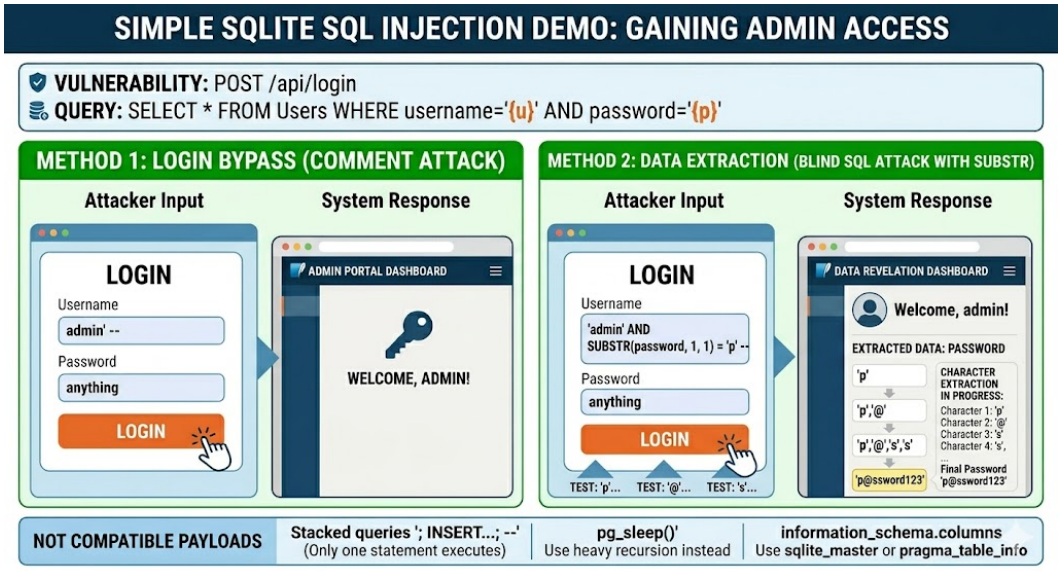

Figure 2: Successful SQL injection against the unshielded implementation, showing that the crafted login payload is accepted and query logic is altered.

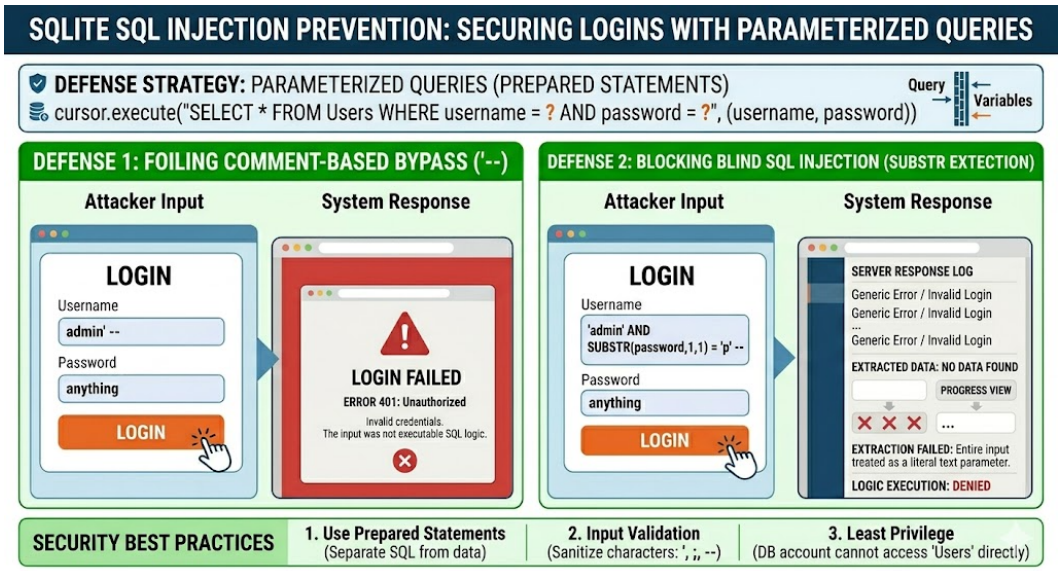

Figure 3: Shielded SQL injection handling with `ENABLE_SQLI_PROTECTION = True`, where the malicious payload is rejected or treated as ordinary input without altering query structure.

a structural restriction: only a single read-only `SELECT` or `WITH` statement is accepted. As a result, mutating statements such as `DROP TABLE`, `UPDATE`, or stacked multi-statement payloads are rejected, while legitimate read-only queries continue to work. This makes the defense more realistic for a free-form admin query feature, where the goal is controlled capability rather than complete removal of functionality.

Some additional payloads were also examined to clarify the limits of the current demonstration. PostgreSQL-specific constructs such as `pg_sleep()`, `version()`, or `information_schema`-based extraction did not succeed against the current vulnerable login path because the project uses SQLite for this experiment. Similarly, a stacked-query payload attempting to append an `INSERT` statement raised an execution error rather than succeeding, because the present `sqlite3.execute()` path does not permit multiple statements in that form. These cases are important because they show that not every SQL injection string is universally portable; however, they do not weaken the main result. The vulnerable system remains exploitable by several practical SQLite-compatible payloads, whereas the shielded system

Table 1: Observed outcome of SQL injection tests after adding toggle-based secure handlers

Payload category	Unshielded system	Shielded system
Comment and tautology login bypass	Successful unauthorized login	Blocked; login fails normally
`UNION`-based data extraction	User data and query-controlled rows returned	Blocked; no data disclosure
`UNION`-based role forgery and schema manipulation	Forged or metadata-oriented rows can be returned	Blocked; payload treated as data
Injected usernames in student lookups	Result set can be broadened beyond intended user	Blocked; no unauthorized rows returned
Injected course codes in admin delete/search paths	Query behavior can be altered	Payload treated literally; intended row scope preserved
Injected values in enroll/grade/create operations	May alter query semantics in vulnerable flow	Stored or rejected as plain data without execution
Mutating or stacked admin SQL	Can execute directly on vulnerable path	Rejected unless it is a single read-only query

prevents the tested SQL injection attempts while preserving normal authentication behavior.

5 Distributed Denial of Service

5.1 Study

Next, we investigate an application-level distributed denial-of-service scenario centered on repeated login traffic. The `/api/login` endpoint is intentionally costly in the vulnerable configuration because it uses the `handle_student_login()` path, which introduces an artificial delay before issuing the database query. When `ENABLE_DDOS_PROTECTION = False`, every request, including attacker-generated requests with invalid credentials, is allowed to execute this delayed path. Under sustained flooding, these attacking login attempts consume backend worker capacity and cause legitimate users to experience heavy slowdown.

In the shielded configuration, the same login endpoint and user-facing workflow are preserved, but a DDoS protection layer is enabled around the application. In the case of repeated attacks from a fixed IP Address, a rate limiting mechanism is used, that blocks any further than 10 requests a second, and 100 requests a minute. This prevents any Denial of Service attack from a single IP Address. In the case of a Distributed attack, the attack script continuously sends randomized login attempts and includes spoofed `X-Forwarded-For` headers to imitate varying source IPs. In the current local deployment, the middleware treats such client-supplied forwarded headers as untrusted and rejects those requests before the expensive login logic is reached. Legitimate browser traffic does not include this header and therefore continues through the normal path. In addition, the shielded login flow places a bounded semaphore around expensive authentication work, preventing bursts from creating unbounded concurrent load. The comparison is intended to show that, without protection, attacker traffic is able to consume the costly login path, whereas with protection enabled, spoofed requests are blocked early enough to preserve normal responsiveness for genuine users.

Demonstration setup. The demonstration can be reproduced locally by launching the backend, frontend, and attack script from their respective directories in different terminals:

```
Terminal 1:
cd backend
uvicorn app:app -host 127.0.0.1 -port 8000
Terminal 2:
cd frontend/frontend
```

```
npm start
Terminal 3:
cd ddos_attack
python3 ddos_simulator.py
```

With `ENABLE_DDOS_PROTECTION = False`, the attacker traffic reaches the delayed login path and the frontend becomes visibly slower during the flood. With `ENABLE_DDOS_PROTECTION = True`, the same spoofed attack requests are rejected in middleware and should no longer reach the expensive login handler; the expected observation is that ordinary frontend interaction remains much smoother even while the attack script is running.

5.2 Results

The implemented defense produces a clear qualitative separation between the vulnerable and shielded modes. In the unshielded run, attacker requests are still processed by the delayed login path and therefore consume time even though they eventually fail authentication. Under sustained flooding, this makes the login experience slower for a real user operating through the frontend. In the shielded run, the spoofed requests are rejected before the delayed login handler is reached, so the backend avoids spending that artificial database cost on attacker traffic.

This difference is consistent with the present implementation. The attacker script deliberately sends `X-Forwarded-For` headers, while the local middleware rejects such headers as untrusted. Consequently, in the shielded configuration the dominant attacker-facing outcome is early rejection, and the normal browser path remains separated from the spoofed flood. The protection is therefore not merely reducing the number of successful logins by the attacker; it is preserving availability by preventing malicious requests from reaching the expensive part of the authentication flow.

Table 2: Observed qualitative behavior of the DoS demonstration.

Mode	Attacker request path	Typical attacker outcome	Observed user experience
Unshielded (`ENABLE_DDOS_PROTECTION = False`)	Requests reach the delayed login handler	Authentication fails only after backend work is spent	Frontend becomes noticeably slower under sustained attack
Shielded (`ENABLE_DDOS_PROTECTION = True`)	Spoofed requests are blocked in middleware before login execution	Early rejection, typically HTTP 403	Frontend remains substantially more responsive during the same attack

6 Authorization Bypass

6.1 Study

The third attack category studies authorization bypass. Because the ERP has clearly separated user roles, it is important to evaluate whether a user can access functions or records beyond their intended permissions. This includes scenarios such as requesting pages intended for another role, directly invoking restricted backend endpoints, or attempting to retrieve or modify records that should not be accessible under the current identity.

In this part of the project, the backend first performs authentication and then demonstrates the separate question of authorization. After a successful login, the server generates a fresh random bearer token and stores it in an in-memory token table that maps the token to the authenticated user's `username` and `role`. Subsequent requests are therefore not allowed to declare their role directly through client-controlled headers; instead, the backend extracts the bearer token, looks it up on the server, and recovers the

```
pp --reload --host 0.0.0.0 --port 8000
INFO:     Will watch for changes in these directories: ['/home/rcpatne/Doc
uments/Projects/DBSHIELD/backend']
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [94767] using StatReload
INFO:     Started server process [94769]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:37204 - "POST /api/login HTTP/1.1" 200 OK
INFO:     127.0.0.1:37204 - "POST /api/login HTTP/1.1" 200 OK
INFO:     10.208.132.51:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.32.251.146:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.38.222.62:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.29.182.127:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.190.208.204:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.69.57.198:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.119.226.79:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.105.18.205:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.246.0.126:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.85.10.68:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.110.12.234:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.126.179.220:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.45.44.135:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.221.22.111:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.70.176.149:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.86.149.42:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
INFO:     10.180.232.135:0 - "POST /api/login HTTP/1.1" 401 Unauthorized
```

```
thon ddos_simulator.py
============================================================
WARNING: INITIATING INFINITE DDOS ATTACK LOOP
Targeting: http://127.0.0.1:8000/api/login
Workers: 8 | Batch size: 20 | Pause: 0.05s
============================================================
Press Ctrl+C to stop the attack.

[Worker-0] Sent 20 requests in 0.61s | Statuses: {401: 20}
[Worker-1] Sent 20 requests in 0.61s | Statuses: {401: 20}
[Worker-2] Sent 20 requests in 1.11s | Statuses: {401: 20}
[Worker-3] Sent 20 requests in 1.12s | Statuses: {401: 20}
[Worker-4] Sent 20 requests in 1.61s | Statuses: {401: 20}
[Worker-5] Sent 20 requests in 1.62s | Statuses: {401: 20}
[Worker-6] Sent 20 requests in 2.02s | Statuses: {'Error': 20}
[Worker-7] Sent 20 requests in 2.02s | Statuses: {'Error': 20}
[Worker-0] Sent 20 requests in 1.97s | Statuses: {401: 20}
[Worker-1] Sent 20 requests in 1.98s | Statuses: {401: 20}
[Worker-2] Sent 20 requests in 1.97s | Statuses: {401: 20}
[Worker-3] Sent 20 requests in 1.97s | Statuses: {401: 20}
[Worker-4] Sent 20 requests in 1.97s | Statuses: {401: 20}
[Worker-5] Sent 20 requests in 1.97s | Statuses: {401: 20}
[Worker-6] Sent 20 requests in 2.00s | Statuses: {'Error': 20}
[Worker-7] Sent 20 requests in 2.00s | Statuses: {'Error': 20}
^C
[ATTACK ABORTED] Shutting down simulation...
Attack terminated.
```

Figure 4: Unshielded login endpoint during the spoofed request flood. With `ENABLE_DDOS_PROTECTION = False`, attacker requests still reach the delayed authentication path and are rejected only after backend work is spent, producing many HTTP 401 responses and noticeably degrading responsiveness for legitimate users.

```
INFO:     10.165.51.244:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.140.238.192:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.142.188.188:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.214.4.247:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.87.23.68:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.249.127.234:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.11.68.8:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.181.157.217:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.102.93.236:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.70.95.135:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.246.209.230:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.88.84.136:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.162.171.176:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.83.144.110:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.239.115.171:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.40.212.71:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.42.32.114:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.246.120.113:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.253.88.155:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.91.255.54:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.49.62.96:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     127.0.0.1:55422 - "POST /api/login HTTP/1.1" 200 OK ←
INFO:     10.101.101.114:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.13.169.113:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.229.76.202:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.134.23.217:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.104.11.184:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.103.163.66:0 - "POST /api/login HTTP/1.1" 403 Forbidden
INFO:     10.23.130.235:0 - "POST /api/login HTTP/1.1" 403 Forbidden
```

```
ck$ python ddos_simulator.py
============================================================
WARNING: INITIATING INFINITE DDOS ATTACK LOOP
Targeting: http://127.0.0.1:8000/api/login
Workers: 8 | Batch size: 20 | Pause: 0.05s
============================================================
Press Ctrl+C to stop the attack.

[Worker-0] Sent 20 requests in 0.05s | Statuses: {403: 20}
[Worker-1] Sent 20 requests in 0.06s | Statuses: {403: 20}
[Worker-2] Sent 20 requests in 0.06s | Statuses: {403: 20}
[Worker-3] Sent 20 requests in 0.07s | Statuses: {403: 20}
[Worker-4] Sent 20 requests in 0.07s | Statuses: {403: 20}
[Worker-5] Sent 20 requests in 0.08s | Statuses: {403: 20}
[Worker-6] Sent 20 requests in 0.08s | Statuses: {403: 20}
[Worker-7] Sent 20 requests in 0.09s | Statuses: {403: 20}
[Worker-0] Sent 20 requests in 0.01s | Statuses: {403: 20}
[Worker-1] Sent 20 requests in 0.01s | Statuses: {403: 20}
[Worker-2] Sent 20 requests in 0.02s | Statuses: {403: 20}
[Worker-3] Sent 20 requests in 0.02s | Statuses: {403: 20}
[Worker-4] Sent 20 requests in 0.02s | Statuses: {403: 20}
[Worker-5] Sent 20 requests in 0.02s | Statuses: {403: 20}
[Worker-6] Sent 20 requests in 0.02s | Statuses: {403: 20}
[Worker-7] Sent 20 requests in 0.02s | Statuses: {403: 20}
[Worker-0] Sent 20 requests in 0.01s | Statuses: {403: 20}
[Worker-1] Sent 20 requests in 0.01s | Statuses: {403: 20}
[Worker-2] Sent 20 requests in 0.01s | Statuses: {403: 20}
[Worker-3] Sent 20 requests in 0.01s | Statuses: {403: 20}
[Worker-4] Sent 20 requests in 0.01s | Statuses: {403: 20}
```

Figure 5: Shielded login endpoint under the same attack. With `ENABLE_DDOS_PROTECTION = True`, spoofed requests carrying untrusted `X-Forwarded-For` headers are blocked early in middleware, producing HTTP 403 responses, while legitimate browser requests continue to succeed.

associated identity and role from that mapping. This design is useful in the report because it makes the distinction between "logged in" and "allowed to do this action" very explicit.

The unshielded and shielded versions differ in what happens *after* token verification. In both cases, a request must still present a valid token that was issued at login time. In the unshielded mode, selected endpoints intentionally skip the role and ownership checks once the token is accepted, which allows a valid student session to invoke instructor or admin operations or to request another student's records. In the shielded mode, the same token-derived role is checked on the server for every protected endpoint, and student-facing data lookups also verify ownership where required. A runtime flag, `ENABLE_AUTH_BYPASS_PROTECTION`, switches between these two behaviors without changing the external API.

This comparison highlights the core takeaway of the authorization-bypass study: authentication alone is not sufficient. Even when a token correctly represents a real logged-in user, the backend must still verify that the role attached to that token is permitted to perform the requested action and, where necessary, that the requested data belongs to that user.

Testing procedure. For testing, the variable `ENABLE_AUTH_BYPASS_PROTECTION` in the backend is switched between `False` (unshielded authorization behavior) and `True` (shielded authorization behavior). A typical test first logs in as a student to obtain a bearer token and then reuses that token against an endpoint that should require a stronger role. For example:

```
TOKEN=$(curl -s -X POST http://localhost:8000/api/login \
  -H "Content-Type: application/json" \
  -d '{"username":"student1","password":"pass1"}' | jq -r '.token')
```

The expected outcome of this login request is a successful response containing a token together with the authenticated `username` and `role`. That token can then be used to test a protected admin endpoint:

```
curl -s -X POST http://localhost:8000/api/admin/action \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"query":"SELECT username FROM users LIMIT 1"}' | jq .
```

When `ENABLE_AUTH_BYPASS_PROTECTION = False`, the expected output is a successful response such as `{"message":"Admin action executed.", ...}`, even though the token belongs to a student. When `ENABLE_AUTH_BYPASS_PROTECTION = True`, the same request is expected to fail with a controlled authorization error such as HTTP 403 and an `Access denied` message. A second useful test is to call `/api/student/view-grades` with another student's username while logged in as `student1`; the vulnerable mode returns another student's data, whereas the shielded mode returns a denial based on ownership checking.

6.2 Results

The authorization-bypass experiments showed a clear separation between authentication and authorization in the implemented backend. A user could obtain a bearer token only after a successful login at `/api/login`, and the server stored that token in its in-memory token table together with the corresponding `username` and `role`. Later requests therefore used a real authenticated session; the bypass behavior did not come from forged headers or fake identities, but from whether the backend performed role and ownership checks after recovering the session from the token.

In the unshielded mode, the vulnerable behavior remained demonstrable exactly as intended. After logging in as `student1`, the same student token could be reused to call endpoints such as `/api/admin/action` or instructor endpoints like `/api/instructor/assign-grade`, even though those routes are meant for stronger roles. Similarly, a student session could call `/api/student/view-grades` with another student's username and receive data that should not have been exposed. These outcomes confirmed that the vulnerable configuration represents an authorization failure rather than an authentication failure.

In the shielded mode, the same requests were rejected consistently on the server side. The backend extracted the token, looked it up in the server-side mapping, recovered the user's role, and then compared that role against the allowed roles for the endpoint. Requests to administrator and instructor endpoints from a student token therefore failed with controlled HTTP 403 responses, while cross-user grade access was blocked by the ownership check applied to student data. The main result is therefore that the shielded implementation preserves valid login and session handling while preventing a logged-in user from turning that valid session into unauthorized access.

```
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$ TOKEN=$(curl -s -X POST
http://localhost:8000/api/login \
  -H "Content-Type: application/json" \
  -d '{"username":"student1","password":"pass1"}' | jq -r '.token')
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$ echo $TOKEN
CJGv5n8LESupkl4dFVy6FpaF4R9kzYCas8j5iRLAIwc
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$ curl -X POST http://loca
lhost:8000/api/student/view-grades   -H "Content-Type: application/json"   -H "Authorization: Bearer $TOKEN"     -d '{"student_userna
me":"student2"}' | jq .
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   213  100   182  100    31   4553    775 --:--:-- --:--:-- --:--:--  5325
{
  "grades": [
    {
      "course": "CS251",
      "grade": "B-",
      "semester": "Spring 2026"
    },
    {
      "course": "MA101",
      "grade": "C",
      "semester": "Spring 2026"
    },
    {
      "course": "PH101",
      "grade": "A-",
      "semester": "Spring 2026"
    }
  ]
}
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$
```

Figure 6: Successful authorization bypass in the unshielded implementation.

```
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$ TOKEN=$(curl -s -X POST
http://localhost:8000/api/login \
  -H "Content-Type: application/json" \
  -d '{"username":"student1","password":"pass1"}' | jq -r '.token')
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$ echo $TOKEN
3G3PdrfTDon4i45FfQLlHLQ2DUGkNv9aGlhz7g3xIs0
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$ curl -X POST http://loca
lhost:8000/api/student/view-grades   -H "Content-Type: application/json"   -H "Authorization: Bearer $TOKEN"     -d '{"student_userna
me":"student2"}' | jq .
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100   132  100   101  100    31   5736   1760 --:--:-- --:--:-- --:--:--  7764
{
  "detail": "Access denied. Grade viewing requires data ownership. You can only access your own data."
}
(venv) arnav_priyadarshi@Arnav-Priyadarshi-HP:/mnt/c/Study Materials/Semester 6/DBMS Lab/DBSHIELD-/DBSHIELD$
```

Figure 7: Shielded system rejecting the unauthorized request with an HTTP error.

7 Conclusion

This project demonstrated that common database-backed application vulnerabilities can be reproduced clearly in a realistic student ERP setting, and that relatively standard defensive measures substantially improve security without changing the core functionality of the system. By comparing an intentionally vulnerable unshielded version with a hardened shielded version of the same application, we were able to isolate the effect of protection mechanisms across three major attack classes: SQL injection, denial of service, and authorization bypass.

The results show a consistent pattern. In the unshielded system, unsafe SQL construction allowed authentication bypass and unauthorized data access, unsafe request handling made the login workflow vulnerable to service degradation under repeated attack traffic, and missing server-side role and ownership checks enabled authenticated users to access actions and records beyond their permissions. In the shielded system, parameterized queries and restricted admin-query handling prevented the tested SQL injection attacks, middleware and bounded processing preserved responsiveness during the login-flood demonstration, and server-side authorization checks successfully blocked cross-role and cross-user access attempts.

Overall, DBSHIELD shows that database security is not a single control but a combination of safe query handling, controlled resource usage, and strict server-side authorization. The side-by-side design of the project makes this especially clear: when these controls are absent, the same application becomes exploitable in multiple ways, but when they are applied carefully, the system remains functional while resisting the demonstrated attacks. This reinforces the broader lesson that security must be treated as a core design requirement in modern database-driven applications, rather than as an optional add-on.